

AI Textbook Q&A System

A RAG-Based Educational Question Answering System
with Deep Source Tracing & ROS 2 Voice Integration

Hye Ran Yoo (041145212) | Peng Wang (041107730)

CST8507: Natural Language Processing
April 3, 2026

Part 1: RAG Pipeline

Part 2: ROS 2 Voice

1. Introduction & Motivation

Problem > Goal > Our Solution

PROBLEM

Students face 46+ AI/ML textbooks
Hard to navigate across books
No unified search system
No way to verify answer sources

GOAL

Ask natural language questions
Get accurate, sourced answers
Deep source tracing:
Book > Chapter > Page > Region
Click to see original PDF

SOLUTION

RAG pipeline with 4 retrievers
RRF fusion for ranking
Ollama LLM (qwen2.5:0.5b)
React UI + PDF viewer
ROS 2 voice integration

Research Question: How can we build a transparent, traceable educational Q&A system for AI/ML learning?

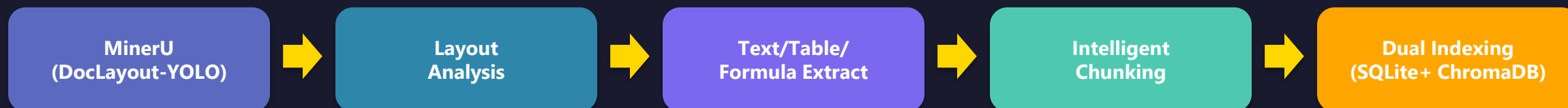
2. Dataset - 46 AI/ML Textbooks

Curated collection of canonical educational resources



46 BOOKS | 85,356 CHUNKS | ~500MB PDFs | Open-access / educational copies

Preprocessing Pipeline



2. Dataset - Library View

Browse all 46 textbooks
with cover art, author, and page count

AI Textbook Q&A
RAG PIPELINE

Search Library

Ddia

Eloquent Javascript

Fastapi Modern Web

Fluent Python

Grl

Http Definitive Guide

Mathematics for Machine Le...

Ml Fundamentals

Nlp

Pattern Recognition and Ma...

Practical Binary Analysis

Pro Git

Probabilistic Machine Learni...

Python Cookbook

Python Testing Pytest

Release It

Serious Cryptography

Snell Probability

Speech and Language Proce...

46
BOOKS

85,356
CHUNKS

Library
46 textbooks in collection

Search by title, author, or keyword...



Practical Binary Analysis

Dennis Andriesse

2018 460 pages



Serious Cryptography

Jean-Philippe Aumasson

2017 314 pages



Bayesian Reasoning and Machine Learning

David Barber

2012 610 pages



SSH, The Definitive Guide

Daniel Barrett et al.

2005 560 pages



TypeScript Deep Dive

Basarat Ali Syed

2020 145 pages



Python Cookbook

Beazley

134 pages



Pattern Recognition and Machine Learning

Christopher M. Bishop

2006 758 pages



Convex Optimization

Boyd

714 pages



Pro Git

Chacon

501 pages



CLRS

Cormen

1312 pages



Mathematics for Machine Learning

Marc Peter Deisenroth et al.

2020 417 pages



Think Python 2e

Downey

244 pages



Think Stats 2e

Downey

264 pages



Nlp

Eisenstein

587 pages



Web Scalability

Ejsmont

417 pages



Art Of Postgresql

Fontaine

457 pages



Sre

Google

550 pages



Swe

Google

602 pages



Http Definitive Guide

Gourley

658 pages



Snell Probability

Grinstead

518 pages



Grl

Hamilton

141 pages



The Elements of Statistical Learning

Trevor Hastie et al.

2009 764 pages



Eloquent Javascript

Haverbeke

435 pages



Speech and Language Processing

Dan Jurafsky & James H. Ma...

2024 626 pages



Ml Fundamentals

Kelleher

853 pages



Ddia

Kleppmann

613 pages



Using Sqlite

Kreibich

528 pages



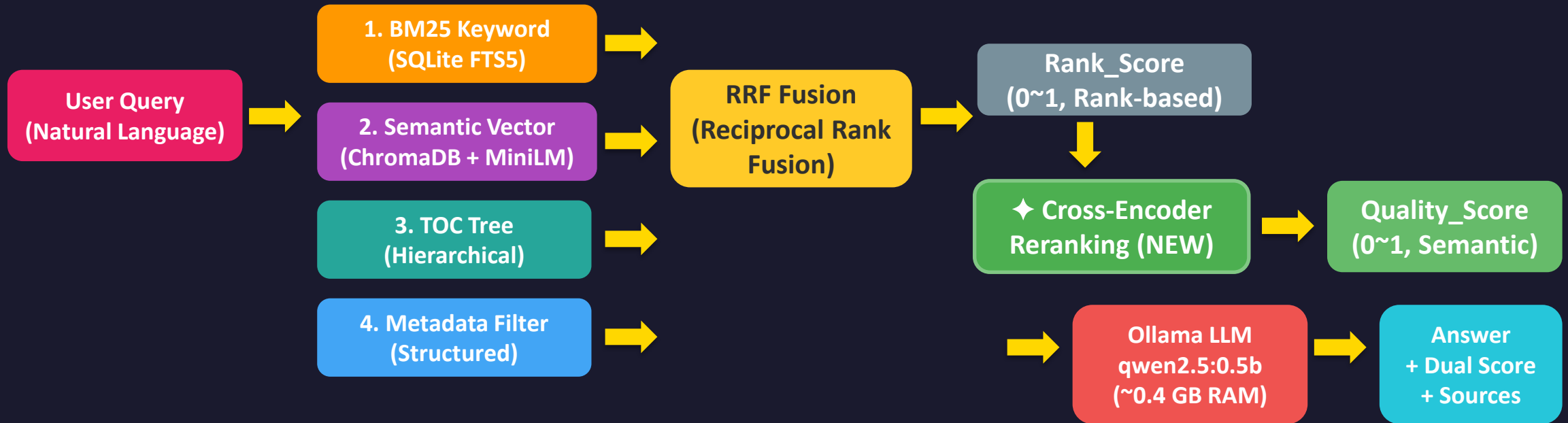
Fastapi Modern Web

Lubanovic

280 pages

3. RAG Architecture – Dual Scoring

4-Method Hybrid Retrieval + RRF (Rank) + Cross-Encoder (Quality)



Dual Scoring System

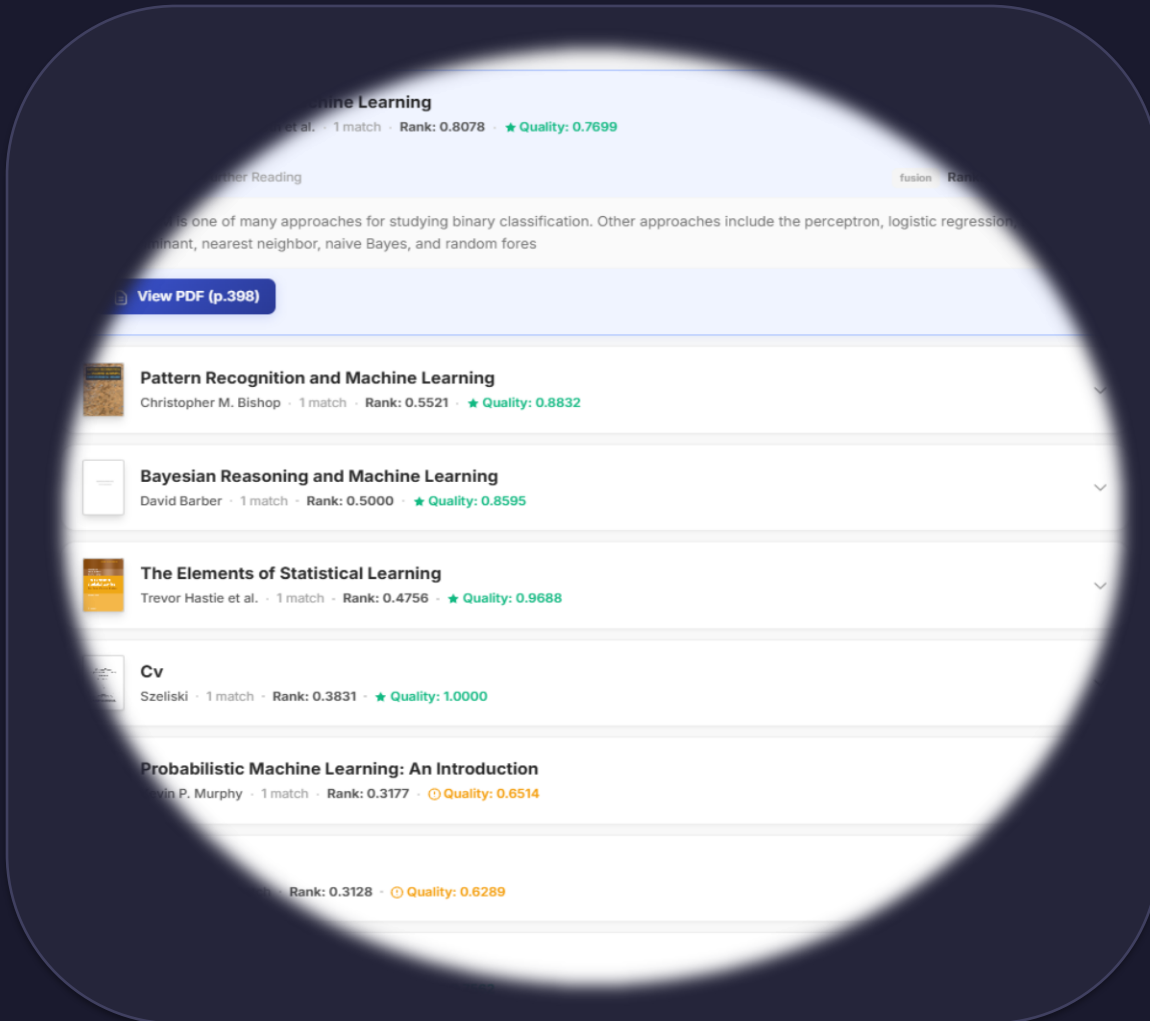
- Rank Score: RRF formula = $\sum(\text{weight} \times 1/(\text{k} + \text{rank}))$, $\text{k}=60$
- Quality Score: Cross-Encoder (query, doc) → relevance (min-max normalized)

Bi-Encoder: all-MiniLM-L6-v2 (Vector Search, 384-dim)

Cross-Encoder: ms-marco-MiniLM-L-6-v2 (Quality Reranking, ~80MB)

3. RAG Architecture

Query: "How does SVM (Support Vector Machine) work?"



Score Interpretation

★ **Quality ≥ 0.70**

High Relevance — Strong match between query and document

⚠ **Quality 0.40–0.69**

Moderate Relevance — Partially related content

✕ **Quality < 0.40**

Low Relevance — Weak or no relevance to query

Scoring Formulas

Rank Score (RRF):

$$\text{score} = \sum \text{weight}_i \times 1 / (60 + \text{rank}_i)$$
 [normalized to 0~1]

Quality Score (Cross-Encoder):

$$\text{score} = \text{CrossEncoder.predict}(\text{query}, \text{document})$$

normalized via min-max within result batch

3. RAG Architecture

Rank-based RRF + Quality-based Cross-Encoder = Better Retrieval

❌ Problem: Rank-Only Scoring

- RRF uses only RANK positions, not actual relevance
- 1st place with score 0.95 = 1st place with score 0.12
- Cannot distinguish truly relevant from coincidentally ranked
- Book dedup may keep wrong chunk if rank is misleading

✅ Solution: Dual Scoring

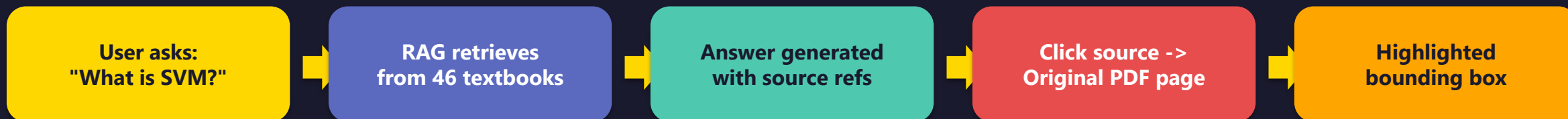
- Cross-Encoder reads (query + document) as a pair
- Produces TRUE semantic relevance score
- Catches high-ranked but irrelevant results
- Zero additional pip install needed (uses sentence-transformers)

Real Example — Query: "How does SVM work?"

Book	Rank Score	Quality Score	Verdict
Mathematics for ML (p.398)	0.8078	0.7688	★ Both high → Truly relevant
Software Engineering (p.xx)	0.1589	0.0000	❌ Low rank + Low quality → Irrelevant
CV (Szeliski)	0.3831	1.0000	★ Lower rank but highest quality → Hidden gem

4. Key Feature - Deep Source Tracing

Not just citations - pixel-level traceability to original documents



What makes this different from basic RAG?

Standard RAG: "According to textbook X, SVM is..."

Our system: Click "View PDF (p.354)" -> opens original page with yellow highlight

Each chunk stores: book_title, chapter, page_number, bounding_box (x, y, w, h)

MinerU (DocLayout-YOLO) extracts spatial coordinates during preprocessing

Enables instant visual verification against original source material

5. React UI – Search & Results & PDF Viewer

AI Textbook Q&A
RAG PIPELINE

SearchLibrary

MODEL
qwen2.5:0.5b

SOURCES: 10

RETRIEVAL
BM25 Keyword
Semantic Vector
TOC Tree
Metadata Filter

LIBRARY COLLECTION
Filter books...
All 46/46
TITLE ^ YEAR ^
Art Of Postgresql
Barto RI Intro
Bayesian Reasoning and Machine Learning
Black Hat Python
Clean Architecture
Clean Code

46 BOOKS85,356 CHUNKS

Ask a Question

Search across 46 AI/ML textbooks with source tracing

What is Optimizer?

Search

Without more context about the user's use case, it's challenging to pinpoint exactly what "Optimizer" means.

SOURCE BOOKS (10)
Probabilistic Machine Learning: An Introduction
Kevin P. Murphy · 1 match · Rank: 0.3333 · Quality: 0.6852
Page 338 · 8.6 Proximal gradient metho...
We are often interested in optimizing an objective of the form
View PDF (p.338)
Bayesian Reasoning and Machine Learning
David Barber · 1 match · Rank: 0.3280 · Quality: 0.7235
Page 348 · 1-norm soft-margin (box con...
to give the optimisation problem:
View PDF (p.348)
Convex Optimization
Boyd · 1 match · Rank: 0.3177 · Quality: 0.5081
CLRS
Cormen · 1 match · Rank: 0.2864 · Quality: 0.4015
Mathematics for Machine Learning
Marc Peter Deisenroth et al. · 1 match · Rank: 0.2785 · Quality: 0.1879
Using SQLite
Kreibich · 1 match · Rank: 0.2711 · Quality: 1.0000
Page 253 · Query Optimization
You can fill out the output section of the structure, providing answers and expense weightings to the optimizer.
View PDF (p.253)
Ydkjs Async Performance

Using SQLite

Page 253 of 528 · Scroll down to go to next page

Communication between the query optimizer and the virtual table is done through the `sqlite3_index_info` structure. This data structure contains an input and output section. The `SQLite` library fills out the input section (input to your function), essentially asking a series of questions. You can fill out the output section of the structure, providing answers and expense weightings to the optimizer.

If `xBestIndex()` sounds complicated, that's because it is. The good news is that if you ignore the optimizer, it will revert to a full table scan for all queries and perform any constraint checking on its own. In the case of the `dblist` module, we're going to take the easy way out, and more or less ignore the optimizer:

```
static int dblist_bestindex( sqlite3_vtab *vtab, sqlite3_index_info *info )  
{  
    return SQLITE_OK;  
}
```

Given how simple the module is, and the fact that it will never return more than 30 rows (there is an internal limit on the number of attached databases), this is a fair trade off between performance and keeping the code simple. Even with fairly large datasets, SQLite does a pretty good job at processing full table scans with surprising speed.

Of course, not all modules—especially internal ones—can get away with this. Most larger modules should try to provide an intelligent response to the optimizer's questions. To see how this is done, we'll take a more in-depth look at this function later on in the chapter. See “[Best Index and Filter](#)” on page 262.

Custom Functions

As much as possible, a good module will attempt to make virtual table instances look and act exactly like standard tables. Functions like `xBestIndex()` help enforce that abstraction, so that virtual tables can interact with the optimizer to correctly produce more efficient lookups—especially in the case of an internal module trying to provide a better or faster indexing method.

There are a few other cases when SQLite needs a bit of help to hide the virtual table abstraction from the database user and other parts of the SQLite engine. SQL function calls and expression processing is one such area.

The `xFindFunction()` allows a module to override an existing function and provide its own implementation. In most cases, this is not needed (or even recommended). The major exceptions are the SQL functions `like()` and `match()`, which are used to implement the SQL expressions `LIKE` and `MATCH` (see [Appendix D](#) for more details).

5. React UI - Feature Details

All features of the interactive search interface

Search Features

- 4 retrieval methods with individual toggles
- Adjustable top-K sources (1-20 slider)
- Library collection filter (select books)
- Model selector (qwen2.5:0.5b)
- Related topic tag suggestions
- Answer with full-text generation
- Source documents with relevance scores

PDF Viewer Features

- Click "View PDF" -> opens inline viewer
- Page navigation (prev/next, page input)
- Zoom controls (fit width, zoom in/out)
- One-page / two-page spread mode
- Highlighted source region (yellow bbox)
- Full-screen viewing mode
- Scroll to navigate through pages

Tech: React (Vite) + FastAPI backend + SQLite FTS5 + ChromaDB + Ollama + pypdfium2

6. Evaluation - 20 Test Questions

Manual assessment across AI/ML/NLP domains

Scoring Method

1.0 = Correct answer with relevant sources

0.5 = Partially correct / incomplete

0.0 = Incorrect or irrelevant

Each question: top 3 retrieved documents

Relevance marked for each source chunk

Final accuracy = mean score across 20 Qs

20 Test Questions

1. What is neural attention?
2. Generative vs discriminative?
3. Gradient for logistic regression?
4. What is an N-gram LM?
5. What is skip-gram?
6. RNNs as language models?
7. 3D rigid body transformation?
8. Weight initialization?
9. Network depth?
10. Cost-sensitive classification?
11. What is TF-IDF?
12. What is backpropagation?
13. Transformer architecture?
14. What is Word2Vec?
15. Named Entity Recognition?
16. What is beam search?
17. Dropout regularization?
18. Batch normalization?
19. What is transfer learning?
20. What is BLEU score?

Result: **69.6%** avg accuracy (target >80%) | Domains: ML, NLP, CV, Math, Deep Learning

Live Demo

 **AI Textbook Q&A**
RAG PIPELINE

Search

Library

MODEL

qwen2.5:0.5b

SOURCES: 11

RETRIEVAL

BM25 Keyword

Semantic Vector

TOC Tree

Metadata Filter

LIBRARY COLLECTION

Filter books...

All 46/46 TITLE ^ YEAR v

TypeScript Deep Dive

Practical Binary Analysis

Serious Cryptography

Bayesian Reasoning and Machine Learning

SSH, The Definitive Guide

Python Cookbook

46

BOOKS

85,356

CHUNKS

AI Textbook Q&A

Search across 46 AI/ML textbooks with source tracing

Q e.g. What is the transformer attention mechanism?

Search

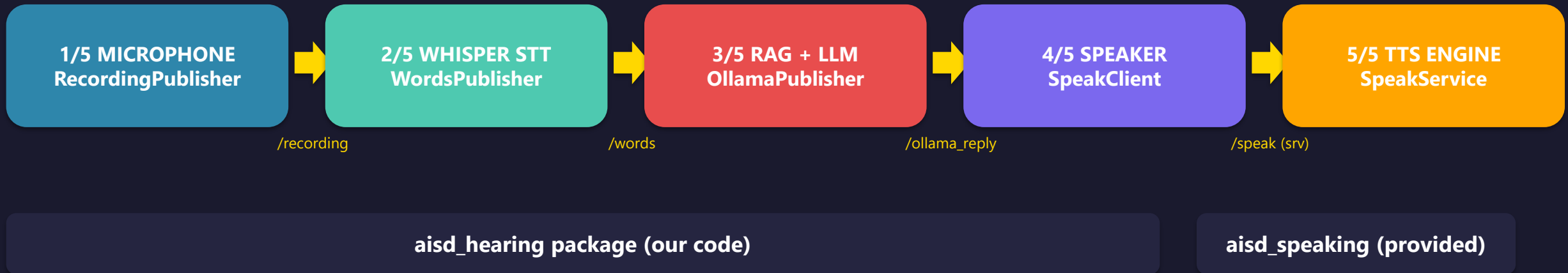
Part 2

ROS 2 Voice Integration

Converting the RAG pipeline into a voice-interactive system
using ROS 2 Humble on Ubuntu 22.04

7. ROS 2 Architecture - 5-Node Pipeline

Voice input -> STT -> RAG -> TTS -> Voice output



Node Details:

1. RecordingPublisher: Mic input + Silero VAD
2. WordsPublisher: faster-whisper (int8, auto-lang)
3. OllamaPublisher: RAG + qwen2.5:0.5b (our node)
4. SpeakClient: Receives reply, calls TTS service
5. SpeakService: gTTS -> audio playback

Deployment:

Platform: Ubuntu 22.04 (loaner laptop)
ROS 2 Humble + colcon build
Process mgmt: tmux (5 panes)
Deploy: SCP from Windows -> laptop
One-click: start_loaner.bat

8. Implementation Details

ollama_publisher.py - The bridge between Part 1 and Part 2

ollama_publisher.py (our new node):

Subscribes to: /words (STT output)

Publishes to: /ollama_reply (LLM answer)

Loads knowledge.txt at startup

Constructs system prompt with RAG context

Queries local Ollama API (localhost:11434)

15s cooldown between replies

Hallucination filtering on input

Deployment Automation:

start_loaner.bat (Windows -> laptop):

Step 1: SCP sync (4 .py + knowledge.txt)

Step 2: Stop existing tmux session

Step 3: colcon build --symlink-install

Step 4: tmux launch 5 panes

stop_loaner.bat:

Kill tmux + cleanup orphan processes

9. ROS 2 Demo - Live Voice Q&A

5-node pipeline running on loaner laptop via tmux

Q&A flow (attention, skip-gram questions) → Continued responses + TTS playback confirmation

Pipeline: Mic → Whisper → RAG + Ollama → gTTS → Speaker

```
Windows PowerShell
ound detected, level: 11378)
[INFO] [1774889418.077818783] [recording_publisher]: [Recording] Stopped: Time limit reached (15s)
[INFO] [1774889418.265702785] [recording_publisher]: [Recording] Finished: 237449 samples captured
[INFO] [1774889418.293709649] [recording_publisher]: [File] Saved: 1774889403014371304.wav
[INFO] [1774889418.294547723] [recording_publisher]: [Recording] Completed: 463.8 KB
[INFO] [1774889418.295476604] [recording_publisher]: [Publish] File published to ROS topic: 1774889403014371304.wav
[INFO] [1774889418.296356202] [recording_publisher]: [Recording] Starting new recording...
[INFO] [1774889418.306161975] [recording_publisher]: [Recording] Waiting for sound input.
.. (speak now)
[INFO] [1774889418.695053777] [recording_publisher]: [Recording] Started (audio level: 7263)

probability distribution over input positions."
[INFO] [1774889389.468102076] [ollama_publisher]: [Words] Received: "What is skip gram in world 2 bag? Yes, the key component in an n-gram language model is the scoring function that calculates attention. Wait using a 2-layer B-forward level and then a..."
[INFO] [1774889389.468835612] [ollama_publisher]: [Ollama] Sending query to local Ollama (qwen2.5:0.5b)...
[INFO] [1774889396.603342265] [ollama_publisher]: [Ollama] Response generated (276 chars)
[INFO] [1774889396.604080373] [ollama_publisher]: [Reply] Published: "Skip-gram is a word embedding method used in Word2Vec to predict surrounding context words given a target word. It uses a two-layer feedforward network with a B (batch size) layer as the encoder and outputs it through an affine transformation with W (weights) as hidden state."
[INFO] [1774889399.686648436] [ollama_publisher]: [Cooldown] Ignoring input (3s < 15s since last reply)
[INFO] [1774889411.494232905] [ollama_publisher]: [Cooldown] Ignoring input (15s < 15s since last reply)

o produce a probability distribution over input positions. This mechanism enables the model to focus on relevant parts of the input when making predictions.": "OK"
[INFO] [1774889335.791559436] [speak_client]: [TTS] Response for "The model uses attention to focus on specific elements of the input data without being overwhelmed by all the details.": "OK"
[INFO] [1774889336.567145599] [speak_client]: [Words] Received: "The key components of an N-gram language model are the scoring function that computes attention weights through a two-layer feedforward network, followed by softmax activation for outputting probabilities."
[INFO] [1774889352.288429263] [speak_client]: [TTS] Response for "The key components of an N-gram language model are the scoring function that computes attention weights through a two-layer feedforward network, followed by softmax activation for outputting probabilities.": "OK"
[INFO] [1774889367.202729827] [speak_client]: [Words] Received: "Yes, the key component in an N-gram language model is the scoring function that calculates attention weights using a two-layer feedforward network and then applying softmax activation to produce a probability distribution over input positions."
[INFO] [1774889386.836035691] [speak_client]: [TTS] Response for "Yes, the key component in an N-gram language model is the scoring function that calculates attention weights using a two-layer feedforward network and then applying softmax activation to produce a probability distribution over input positions.": "OK"
[INFO] [1774889396.606467327] [speak_client]: [Words] Received: "Skip-gram is a word embedding method used in Word2Vec to predict surrounding context words given a target word. It uses a two-layer feedforward network with a B (batch size) layer as the encoder and outputs it through an affine transformation with W (weights) as hidden state."
[INFO] [1774889420.768099281] [speak_client]: [TTS] Response for "Skip-gram is a word embedding method used in Word2Vec to predict surrounding context words given a target word. It uses a two-layer feedforward network with a B (batch size) layer as the encoder and outputs it through an affine transformation with W (weights) as hidden state.": "OK"

[nlp_rag] 0:nodes*
"aisd" 16:50 30-Mar-26
```


Results - Key Achievements

Hybrid RAG

Successfully implemented hybrid RAG architecture with 4 retrieval methods + RRF fusion

Deep Source Tracing

Deep source tracing with clickable PDF references and highlighted bounding box regions

Layout-Aware

Layout-aware document processing preserving tables, formulas, and figures

ROS 2 Integration

Full ROS 2 integration with vocal interaction pipeline (5-node architecture)

Low Memory

Low memory footprint (<1.5GB total) suitable for robot deployment

10. Challenges & Solutions

Whisper Hallucinations

Silero VAD pre-filtering + custom hallucination detector (repetition check, known phrases, trigram analysis)

Echo Loop (TTS to Mic)

15-second cooldown timer after each reply prevents system from hearing its own TTS output

Resource Constraints

qwen2.5:0.5b (0.4GB) + faster-whisper int8 -- 3 AI models (Whisper + Ollama + gTTS) on one CPU laptop

Cross-Platform Deploy

Automated bat/sh scripts: SCP sync -> colcon build -> tmux launch -- one-click deployment from Windows to Ubuntu laptop

PDF Layout Parsing

MinerU + DocLayout-YOLO detects 10 element categories -- preserves tables, formulas, figures with bounding box coordinates

11. Discussion & Future Work

Current Limitations

- Small LLM (0.5B params) - limited generation quality; accuracy 69.6% vs 80% target due to model size constraints
- Single-turn Q&A only (no conversation memory)
- Static knowledge file (no live updates)
- CPU inference is slow (~10-15s per response)
- ROS 2 Part 2 uses simplified knowledge base
- English-only knowledge base content

Future Work

- Upgrade to qwen2.5:3b with GPU support
- Multi-turn conversation with context memory
- Dynamic knowledge base updates via API
- Multi-language support (KR, CN, EN)
- Integration with iRobot Create 3 hardware
- Fine-tuning on educational domain data

Conclusion

We built a RAG-based educational Q&A system that:

- Answers AI/ML questions using 46 canonical textbooks
- Provides deep source tracing to exact PDF pages with highlighted regions
- Implements 4-method hybrid retrieval with RRF fusion
- Dual scoring: RRF rank score + Cross-Encoder quality score for transparent relevance assessment
- Integrates seamlessly with ROS 2 for voice-interactive robot deployment
- Runs locally with low memory footprint (<1.5GB)

A complete voice-interactive RAG pipeline - from 46 textbooks to real-time voice Q&A on a ROS 2 platform.

Thank You!

Questions?

GitHub: github.com/LannieYoo/nlp_project

Python | Ollama | ChromaDB | SQLite | React | FastAPI | ROS 2 | MinerU | Whisper | gTTS